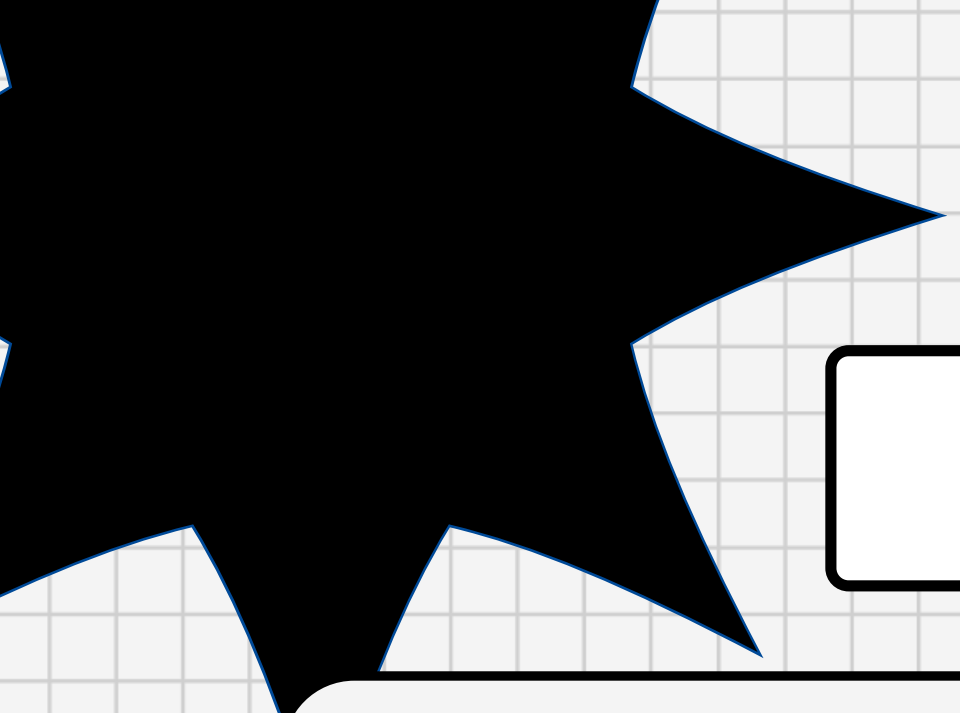
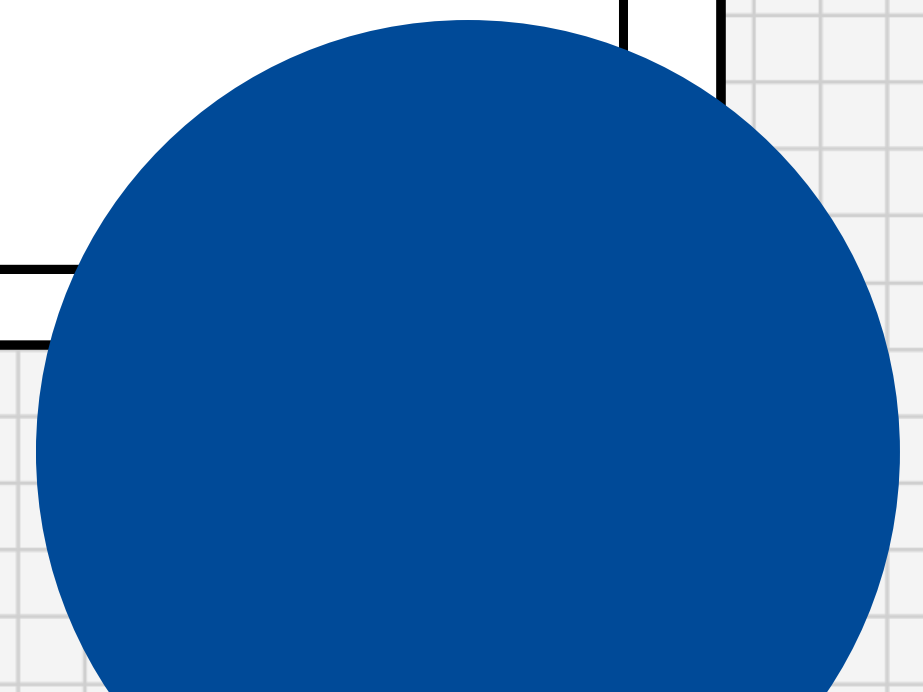




Ayudantía N° 4



InsertionSort y MergeSort



Repaso

Insertion Sort



Version In-place



Input: secuencia A ,
de largo $n \geq 2$

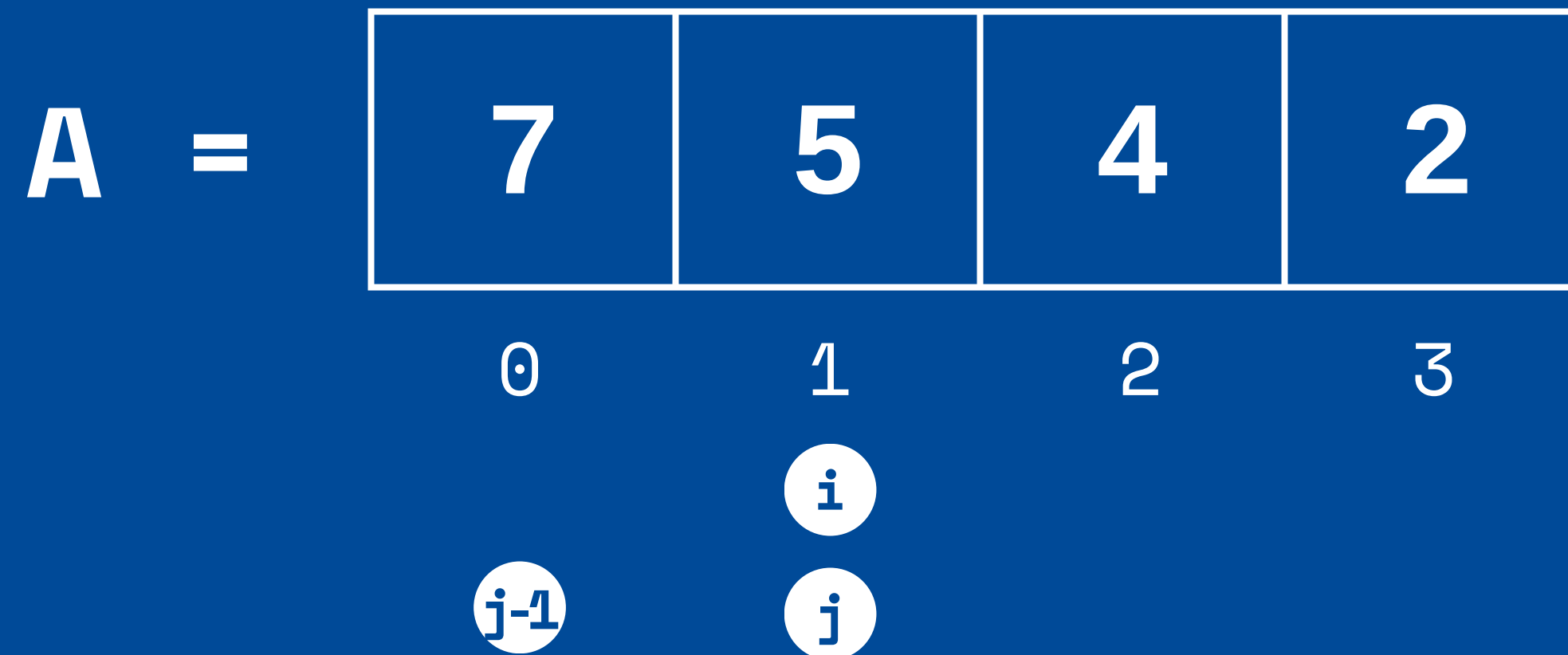
Output: Nada

Visualización

https://www.youtube.com/watch?v=Q1JdRUh1_98&ab_channel=ProfessorPainter

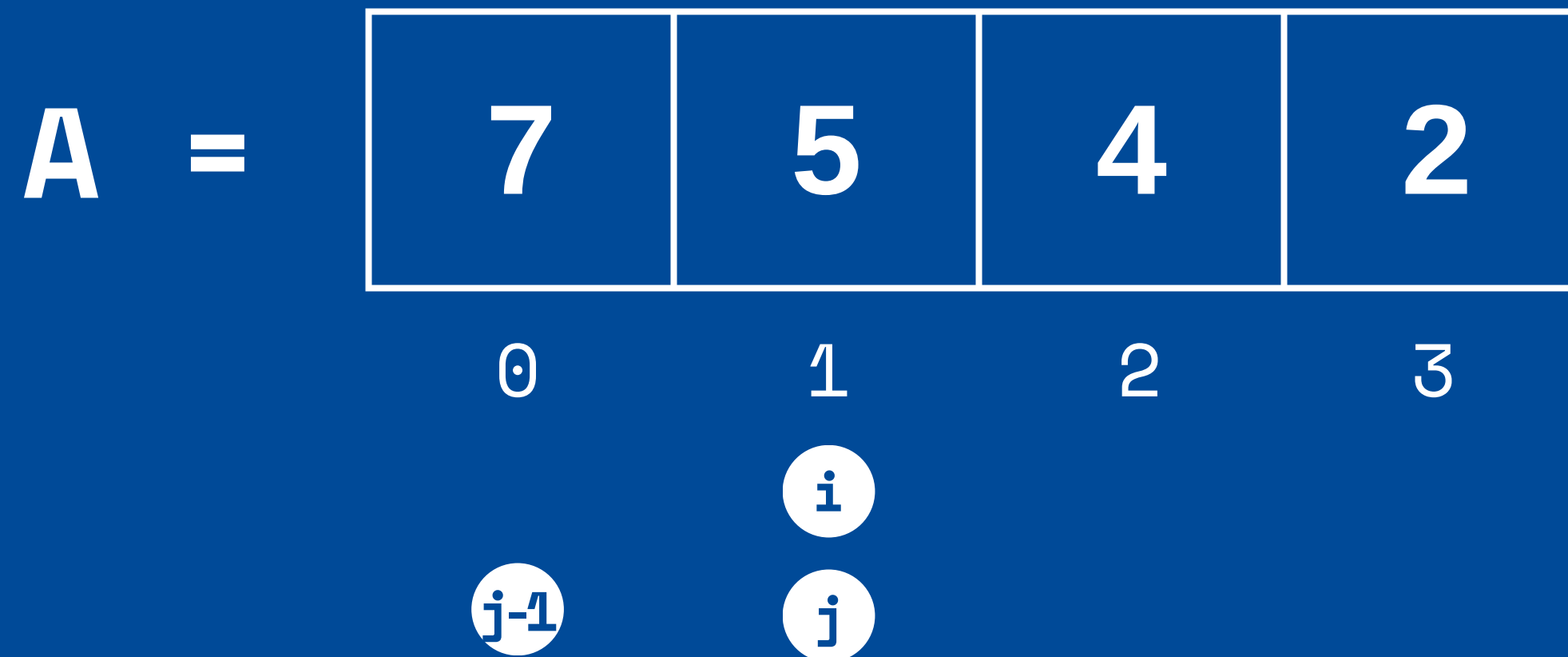
```
InsertionSort ( $A, n$ ):  
1   for  $i = 1 \dots n - 1$ :  
2        $j = i$   
3       while  $(j > 0) \wedge (A[j] < A[j - 1])$  :  
4           Intercambiar  $A[j]$  con  $A[j - 1]$   
5        $j = j - 1$ 
```

insertion sort



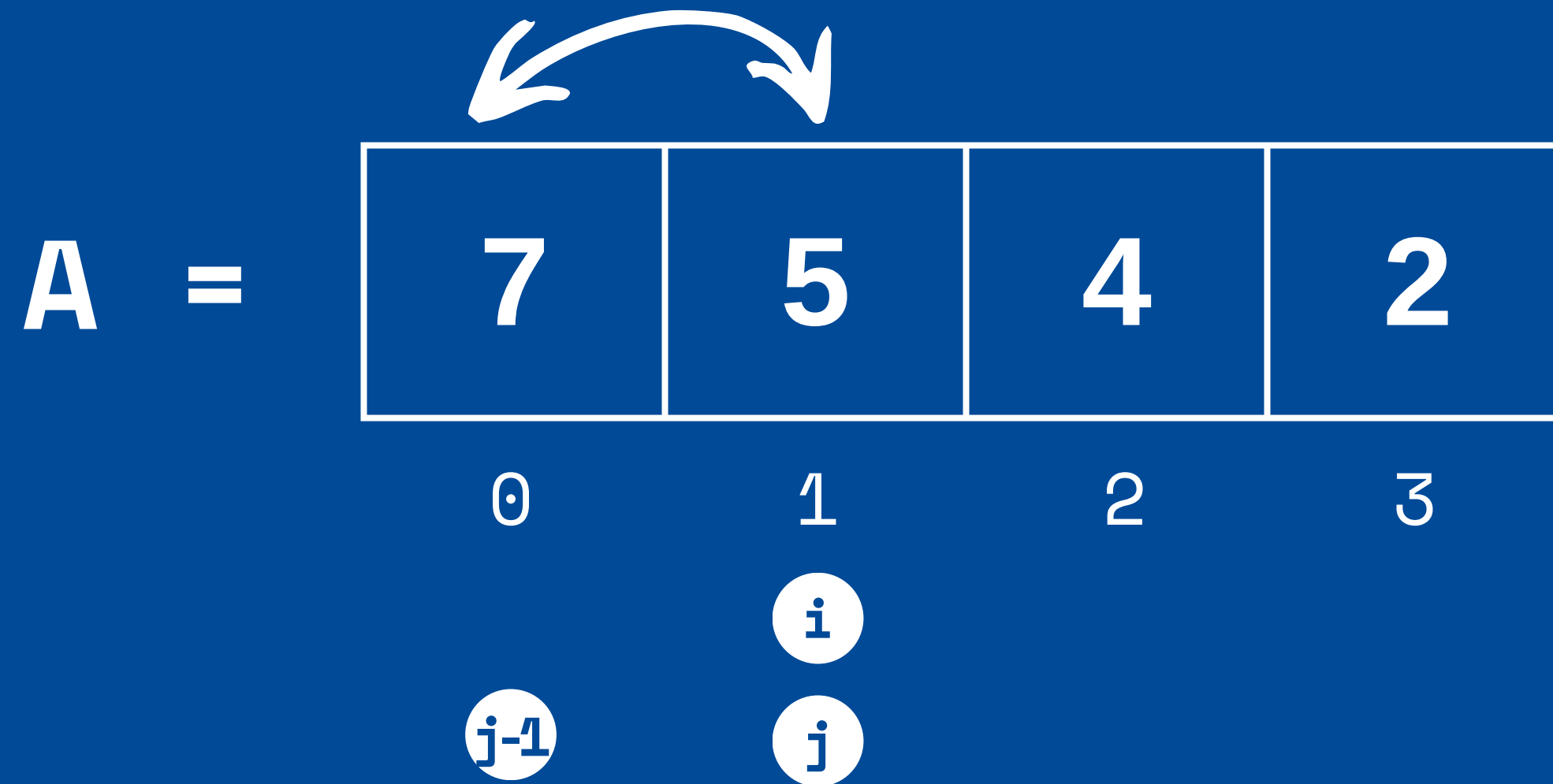
En la 1ª iteración tengo $i=1$ y comenzamos con $j=1$

insertion sort



Es el elemento $A[j] < A[j-1]$?

insertion sort



En este caso $5 < 7$, por lo que los intercambiamos

insertion sort

A =

5	7	4	2
---	---	---	---

0

1

2

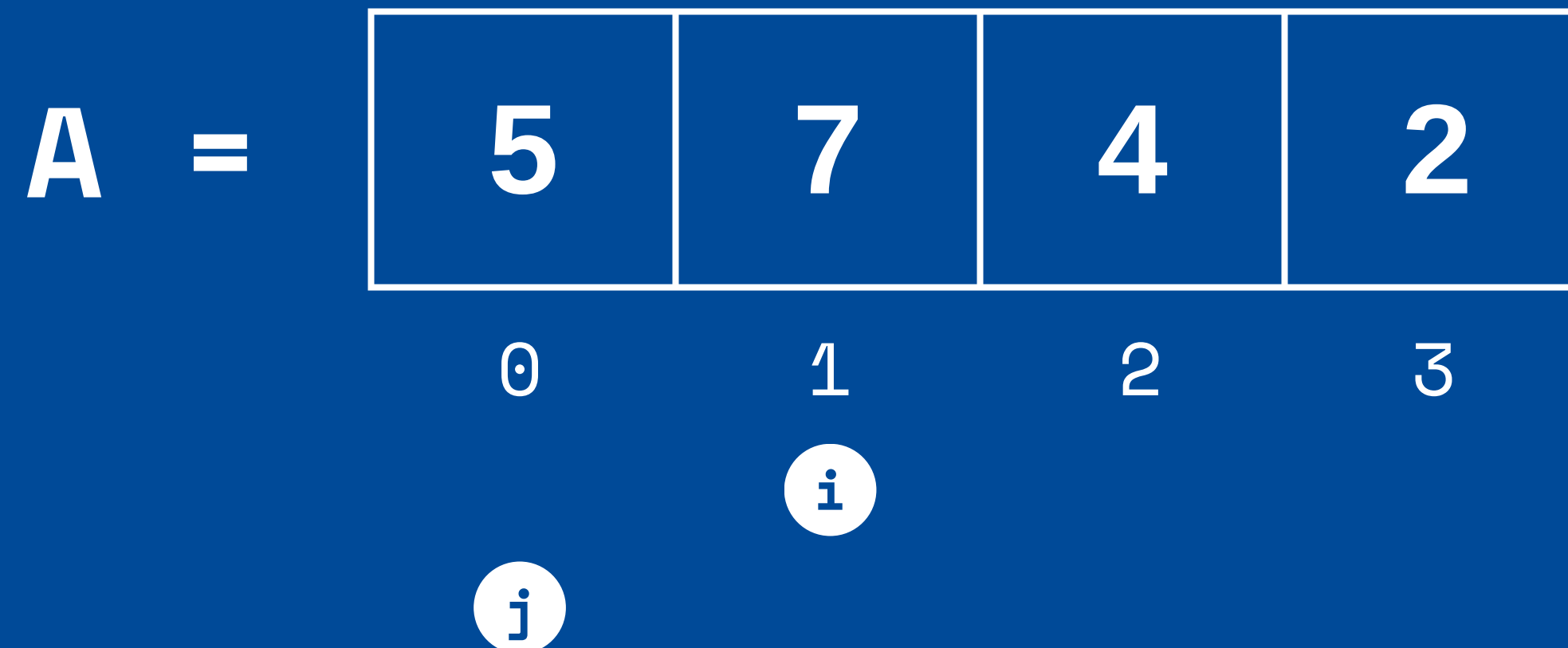
3

i

j-1

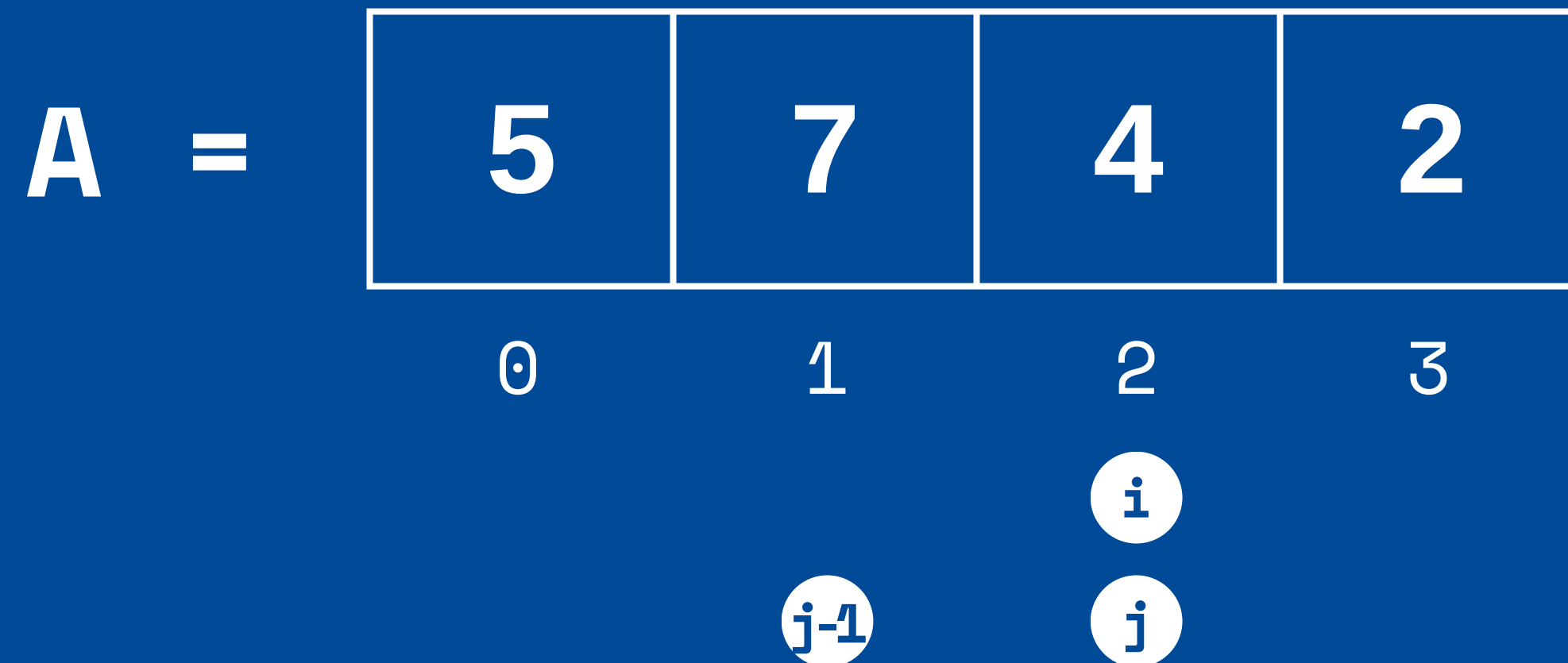
j

insertion sort



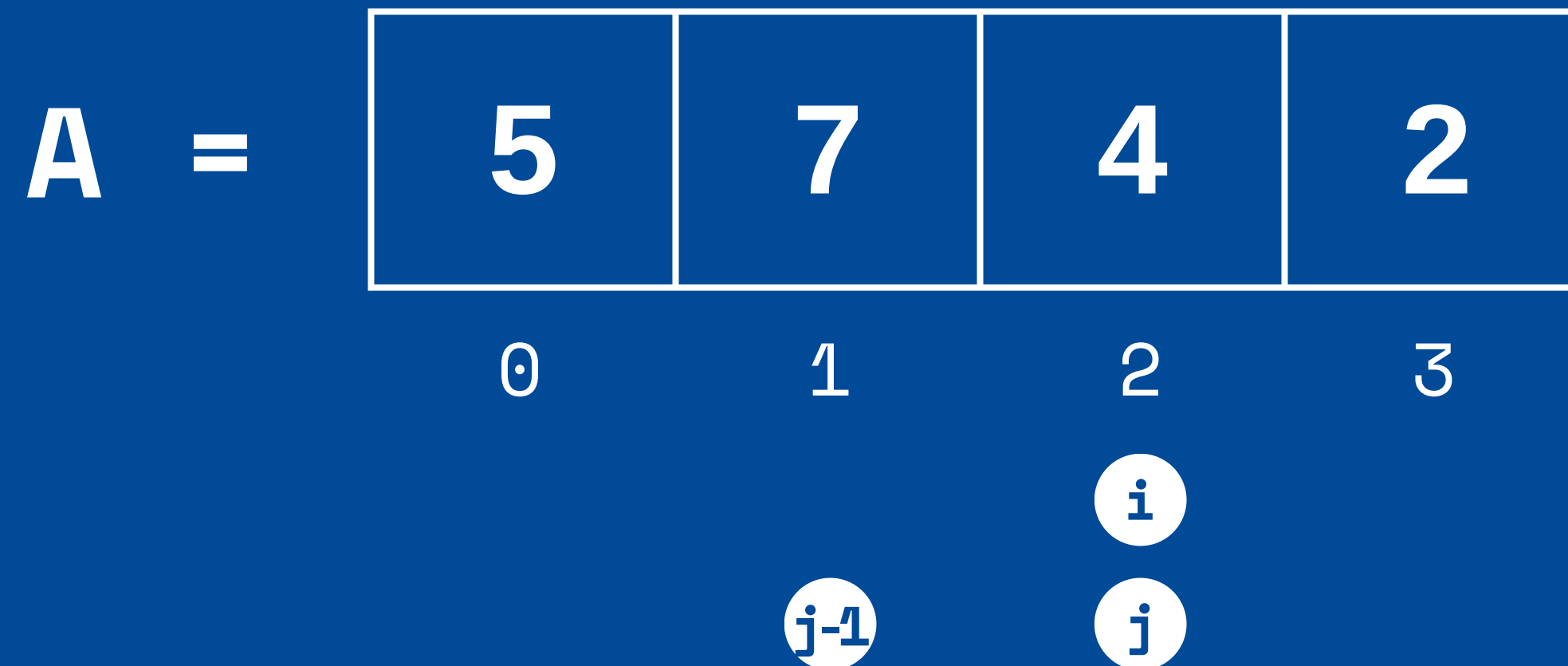
Se actualiza j : $j = j - 1 = 0$, rompiendo el while

insertion sort



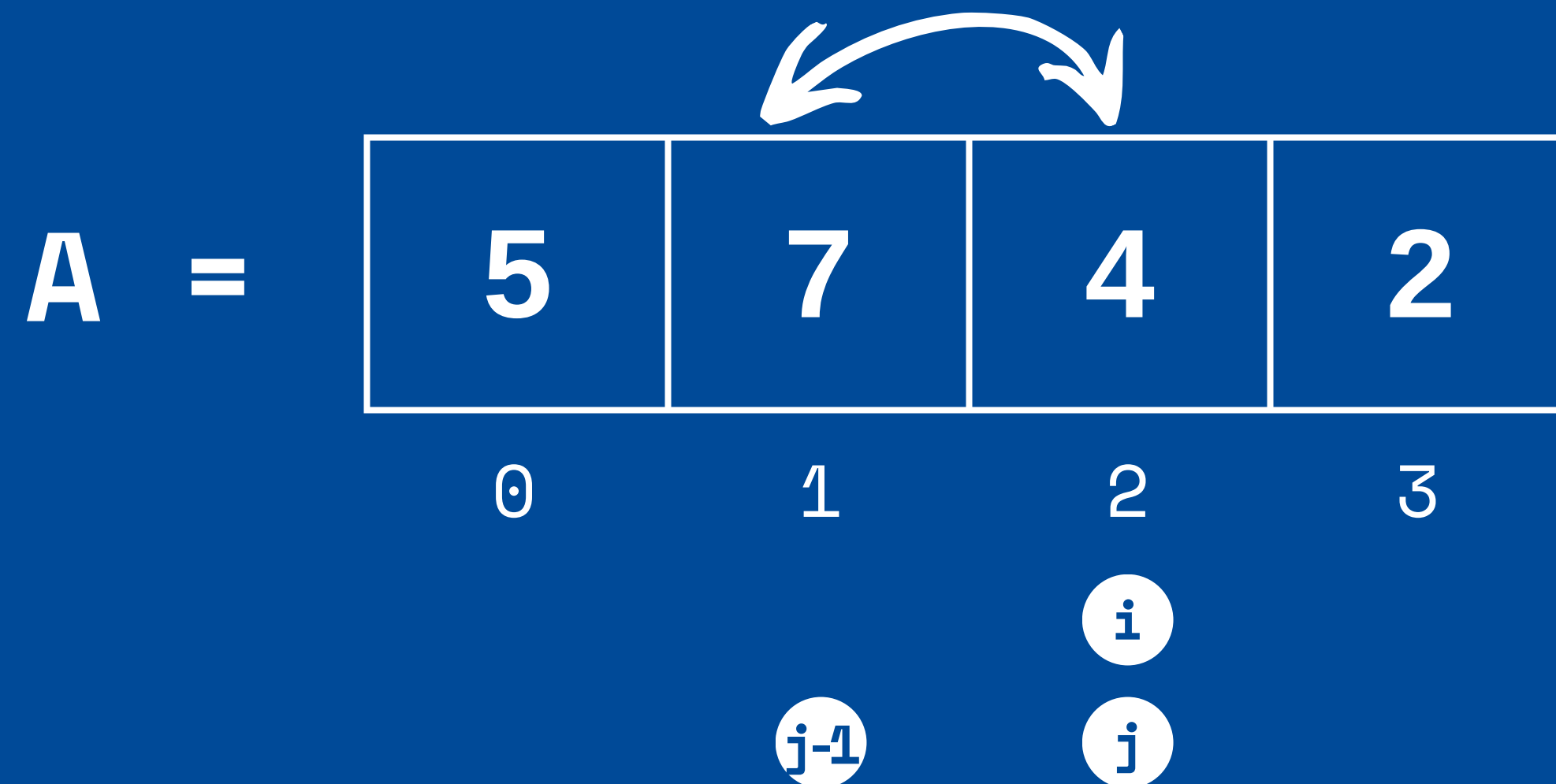
Pasamos a la 2ª iteración, con $i=2$ (y $j = i$)

insertion sort



Es el elemento $A[j] < A[j-1]$?

insertion sort



En este caso $4 < 7$, por lo que los intercambiamos

insertion sort

A =

5	4	7	2
---	---	---	---

0

1

2

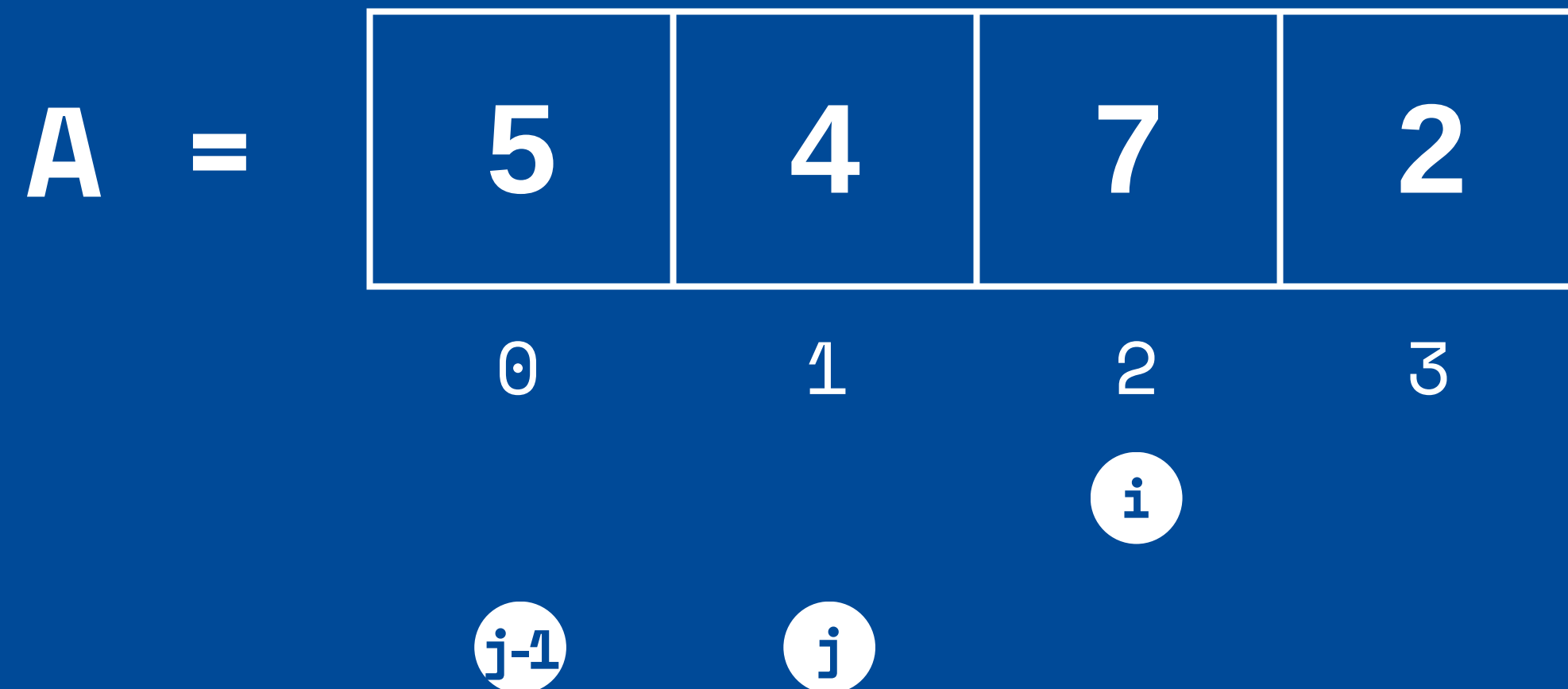
3

i

j-1

j

insertion sort



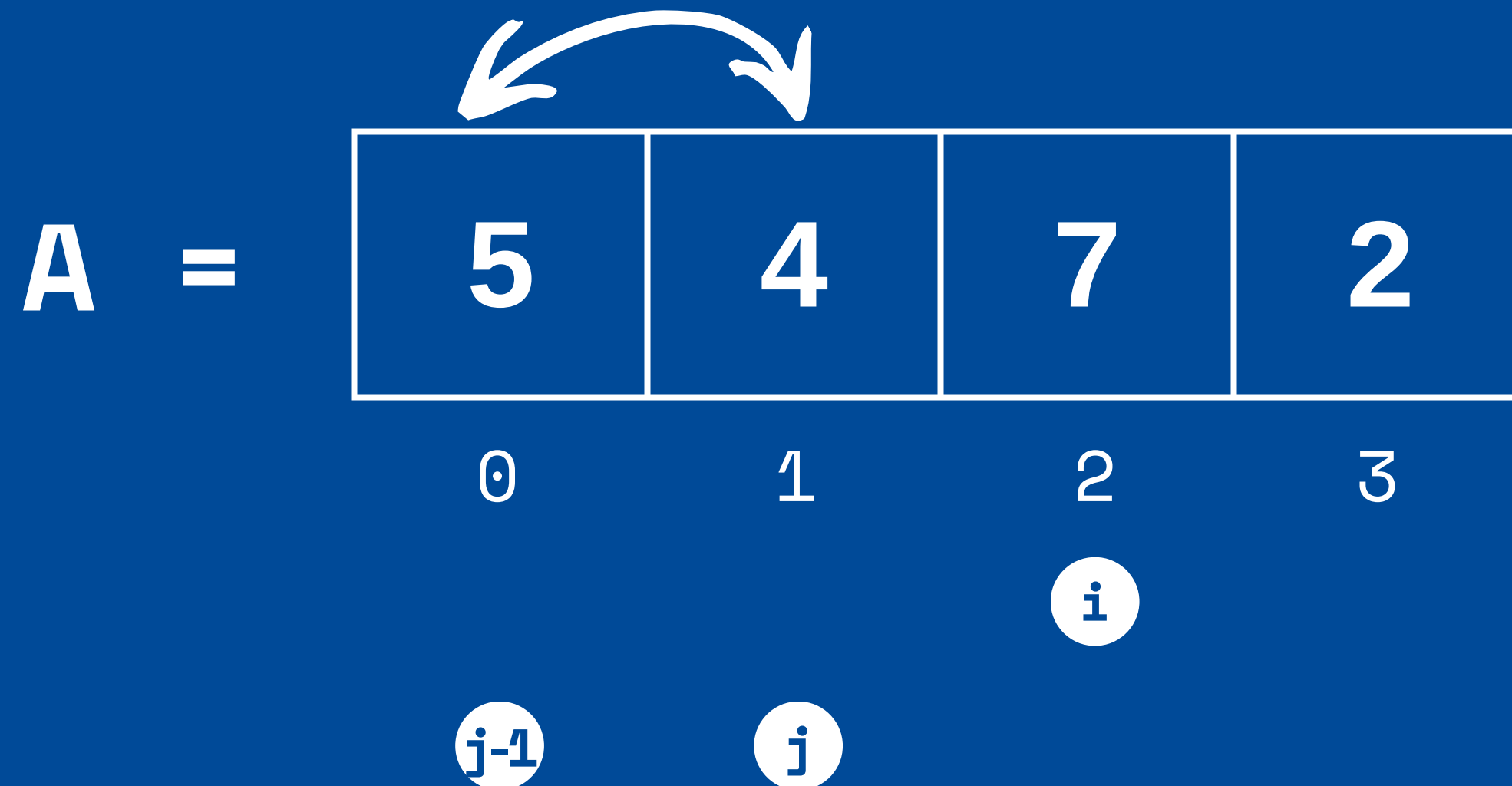
Se actualiza j : $j = j - 1 = 1$

insertion sort



Es el elemento $A[j] < A[j-1]$?

insertion sort



En este caso $4 < 5$, por lo que los intercambiamos

insertion sort

A =

4	5	7	2
---	---	---	---

0

1

2

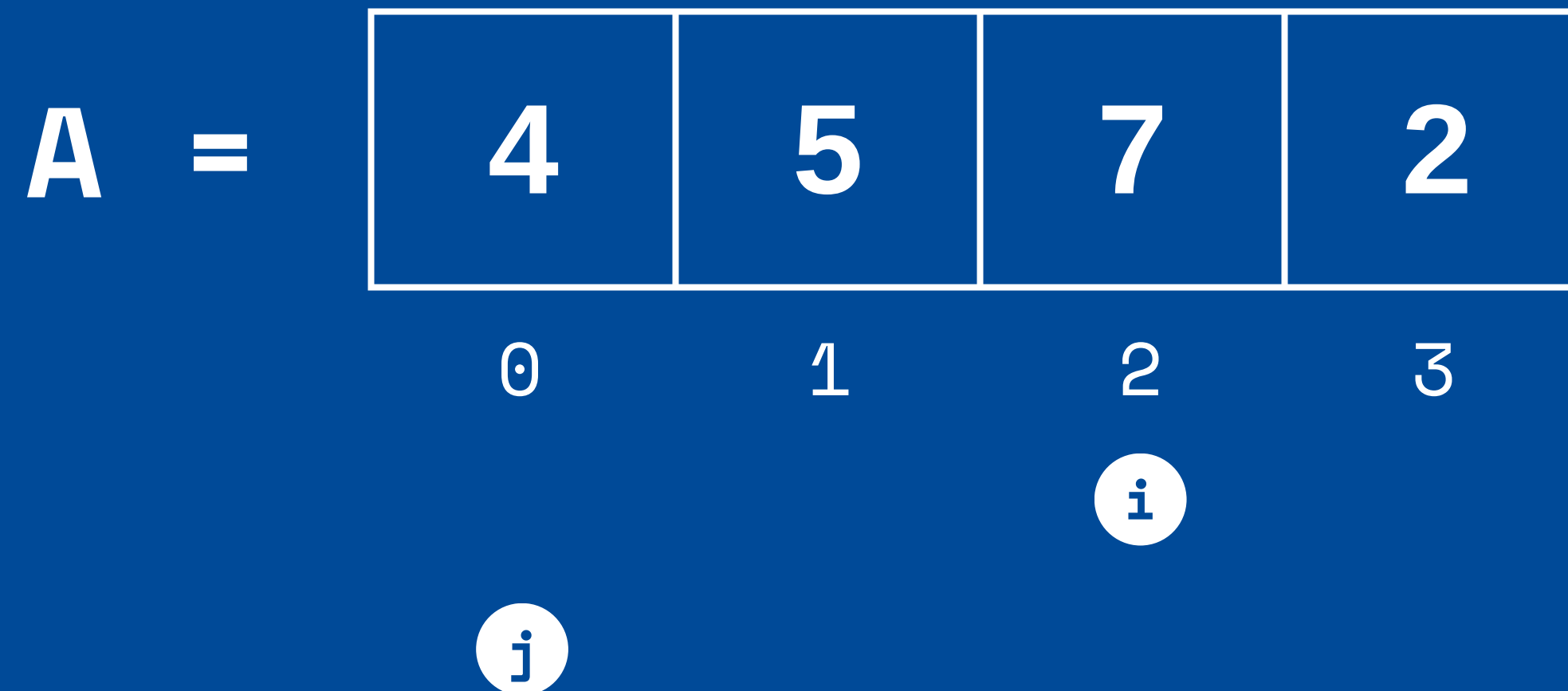
3

i

j-1

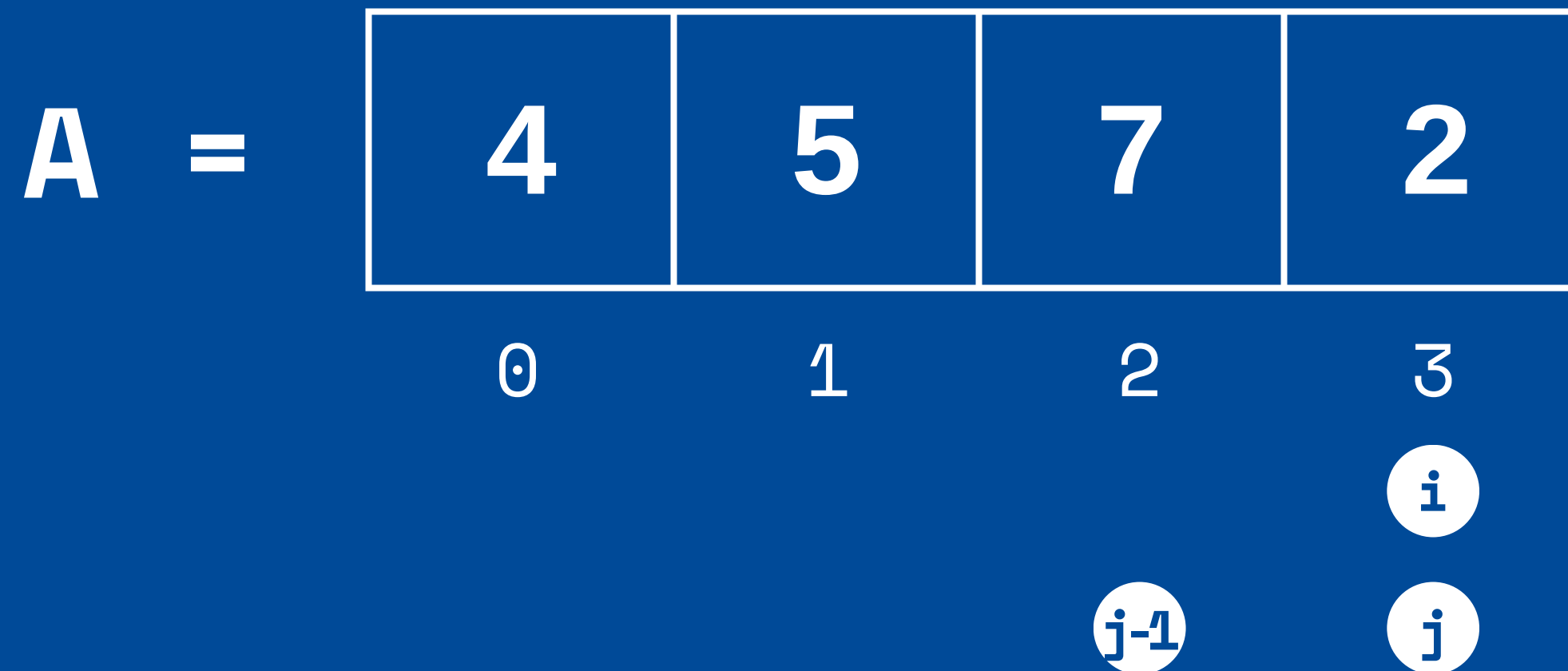
j

insertion sort



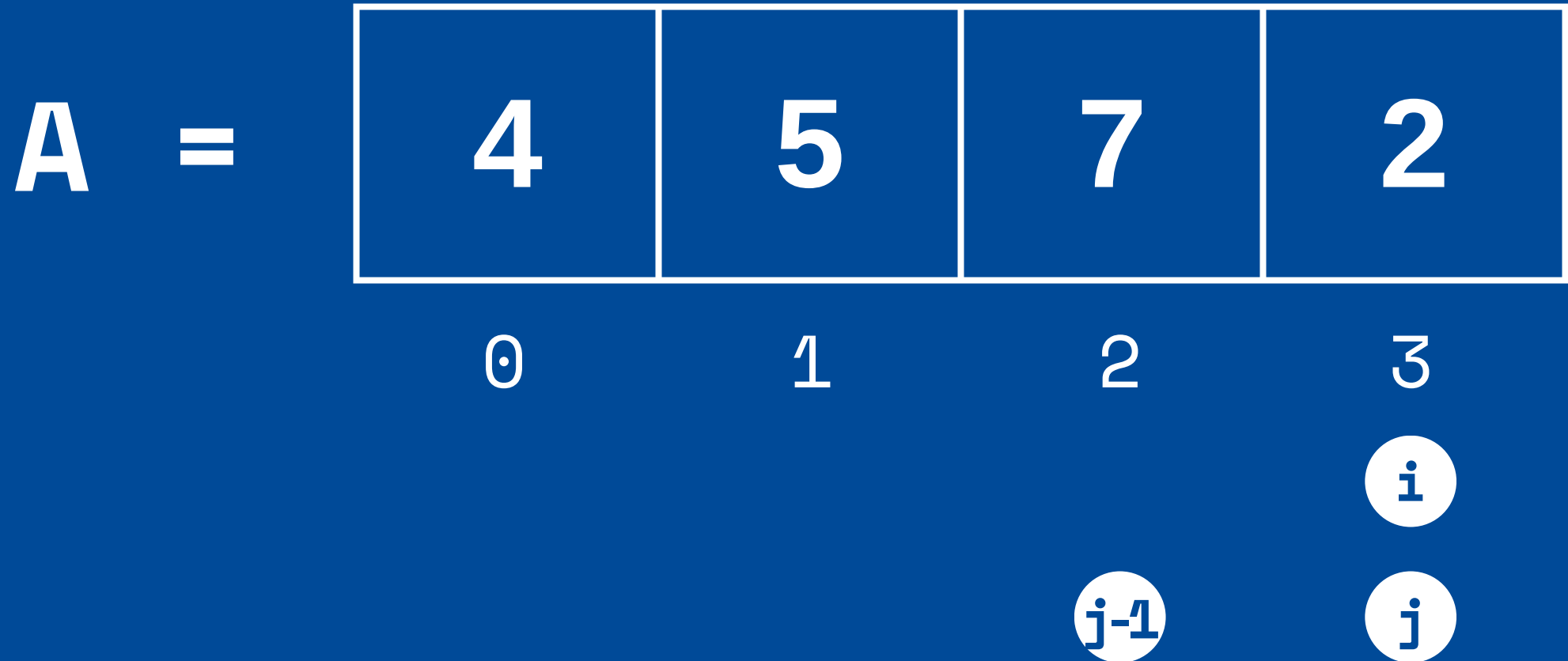
Se actualiza j : $j = j - 1 = 0$, lo que rompe el while

insertion sort



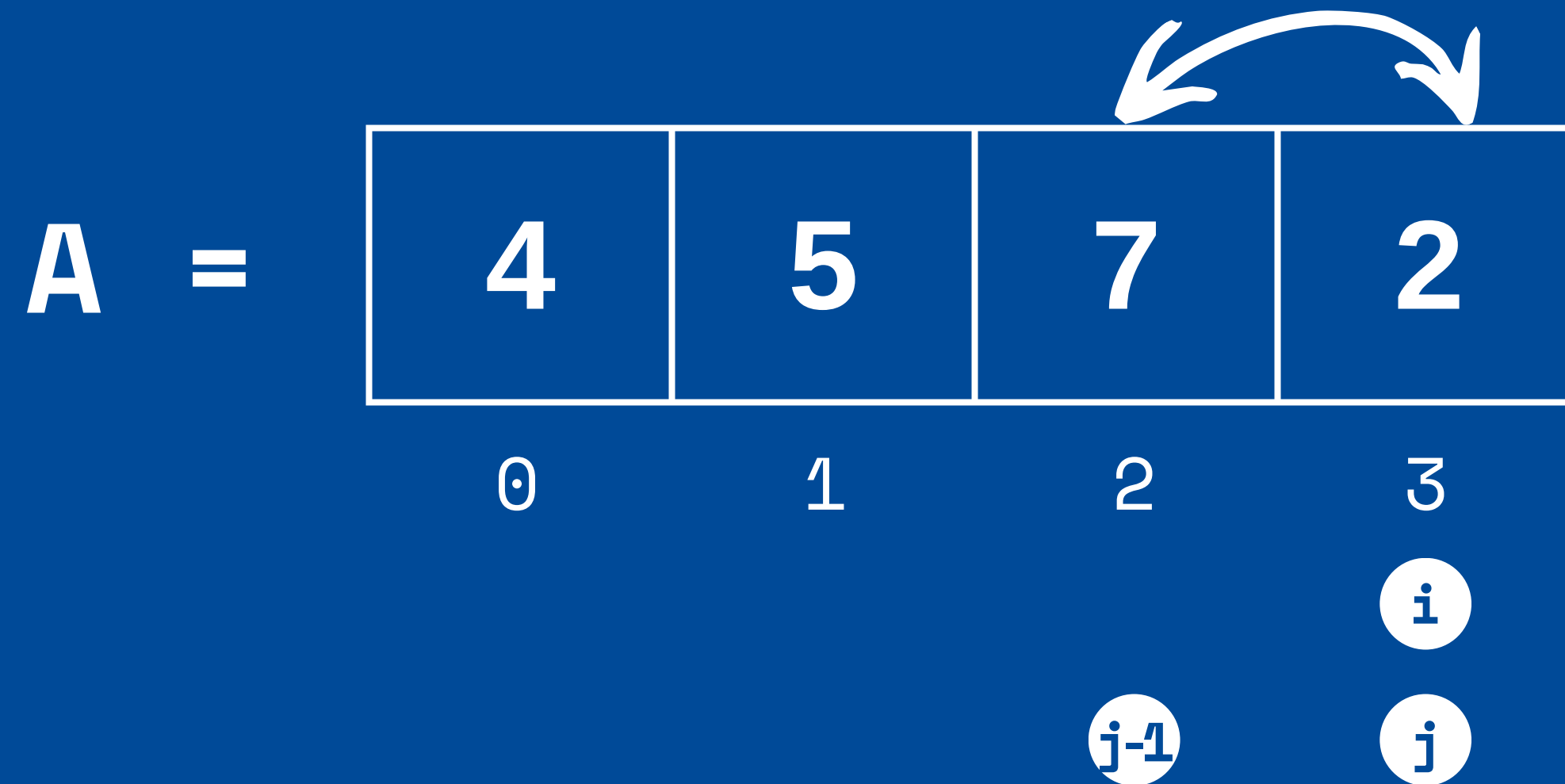
3° iteración, $i = 3$ y $j = i$, notemos que es la última iteración

insertion sort



Es el elemento $A[j] < A[j-1]$?

insertion sort



En este caso $2 < 7$, por lo que los intercambiamos

insertion sort

A =

4	5	2	7
---	---	---	---

0

1

2

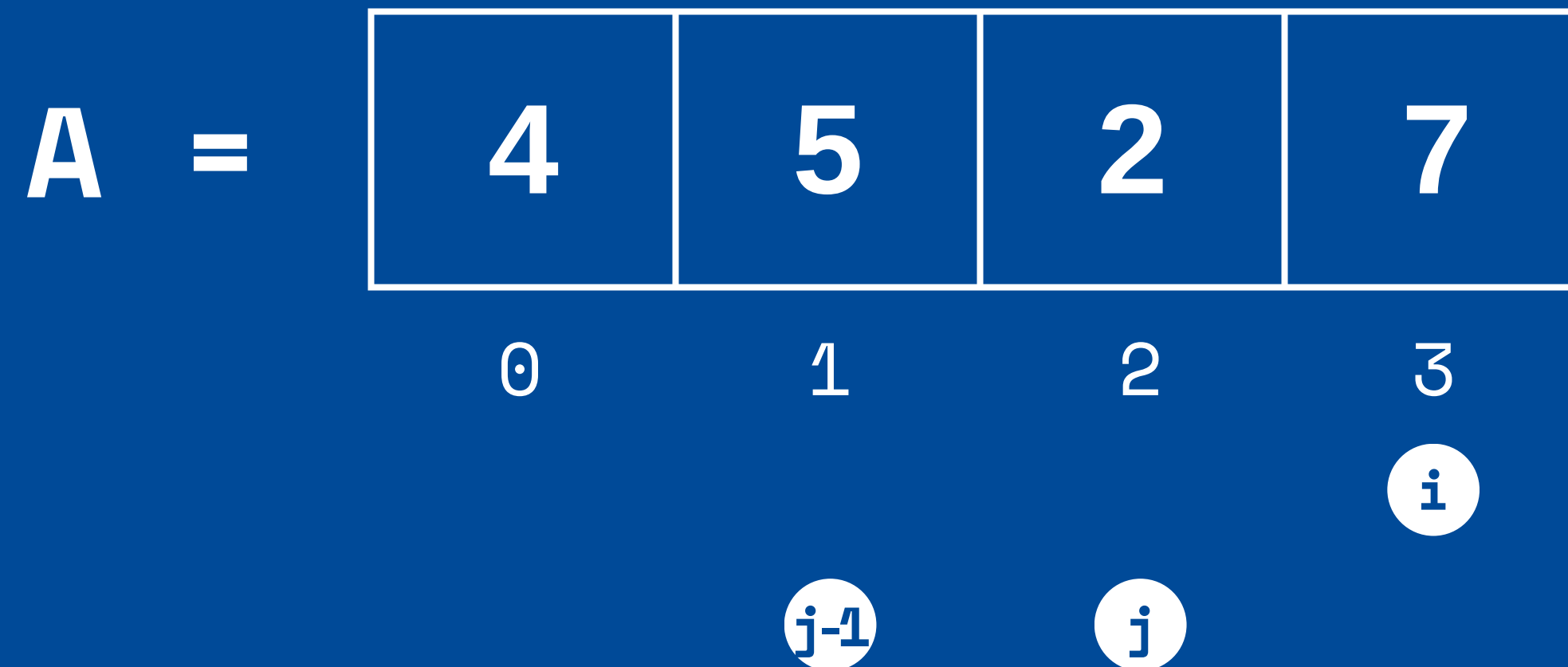
3

i

j-1

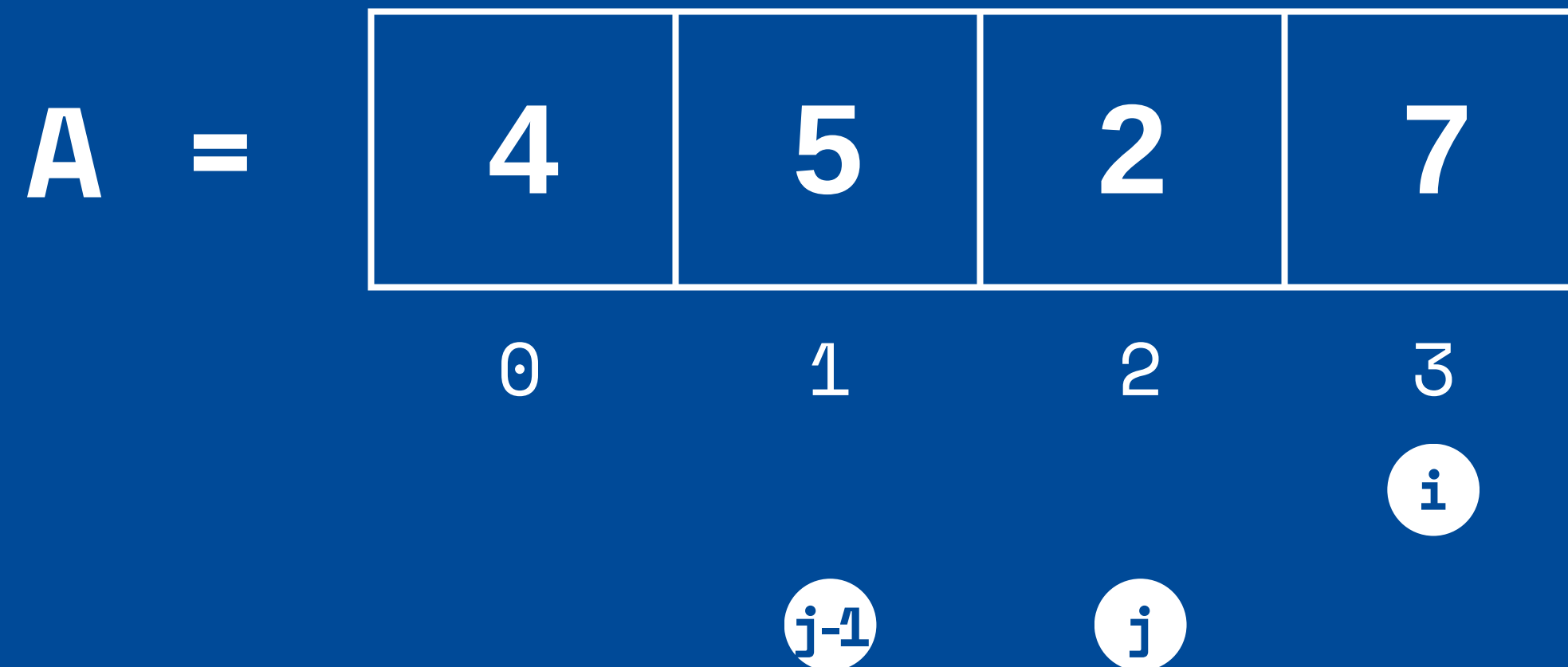
j

insertion sort



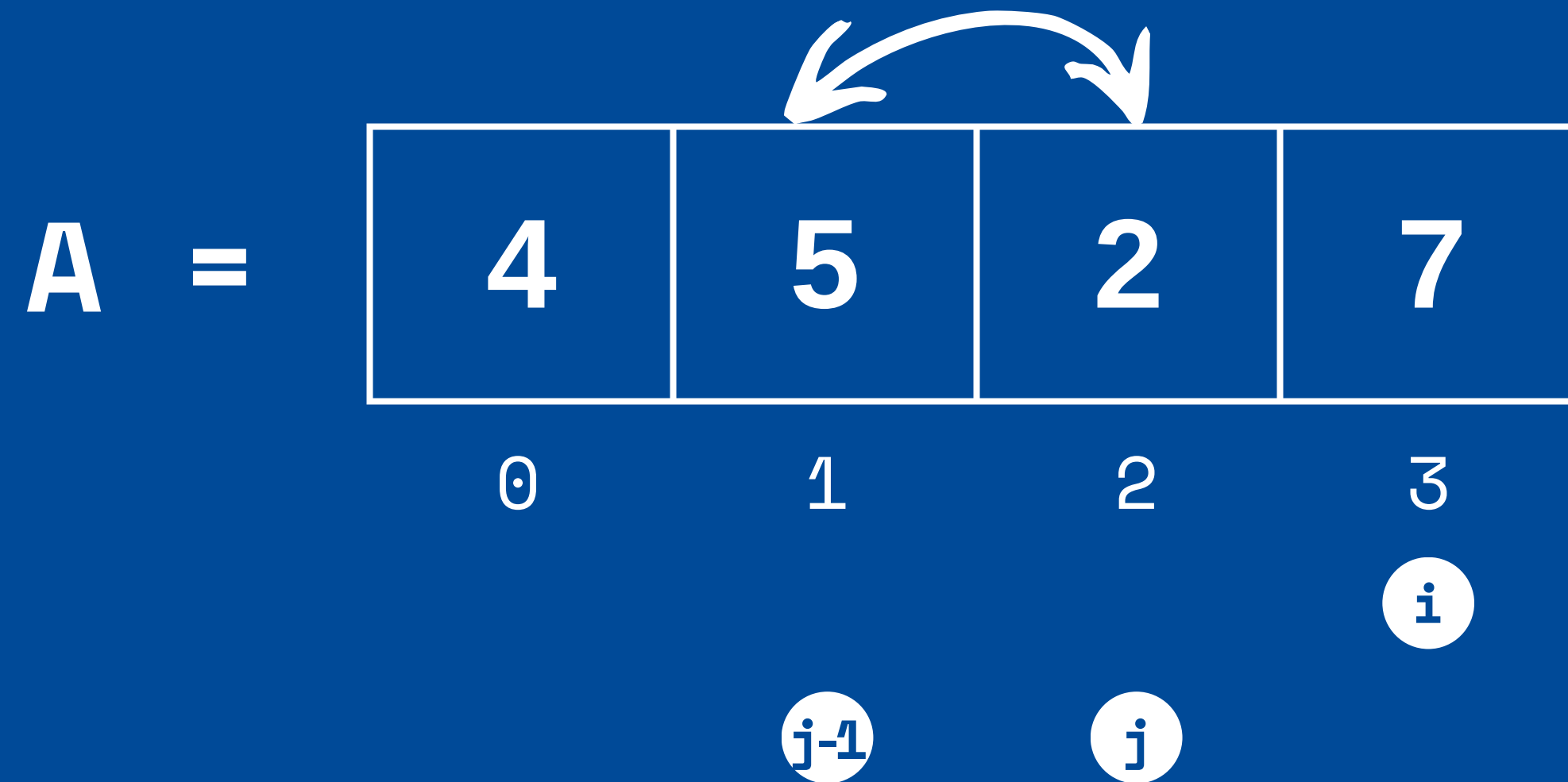
Se actualiza **j**: $j = j - 1 = 2$

insertion sort



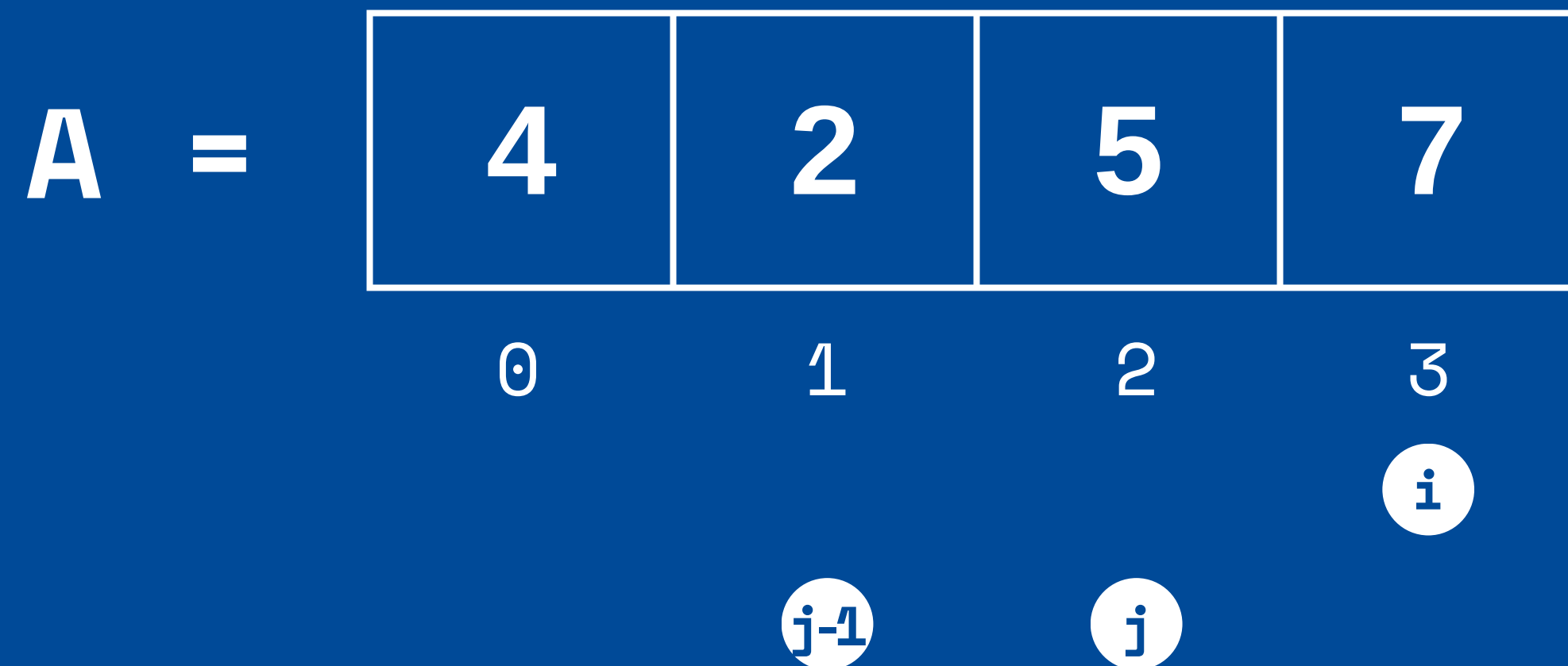
Es el elemento $A[j] < A[j-1]$?

insertion sort



En este caso $2 < 5$, por lo que los intercambiamos

insertion sort

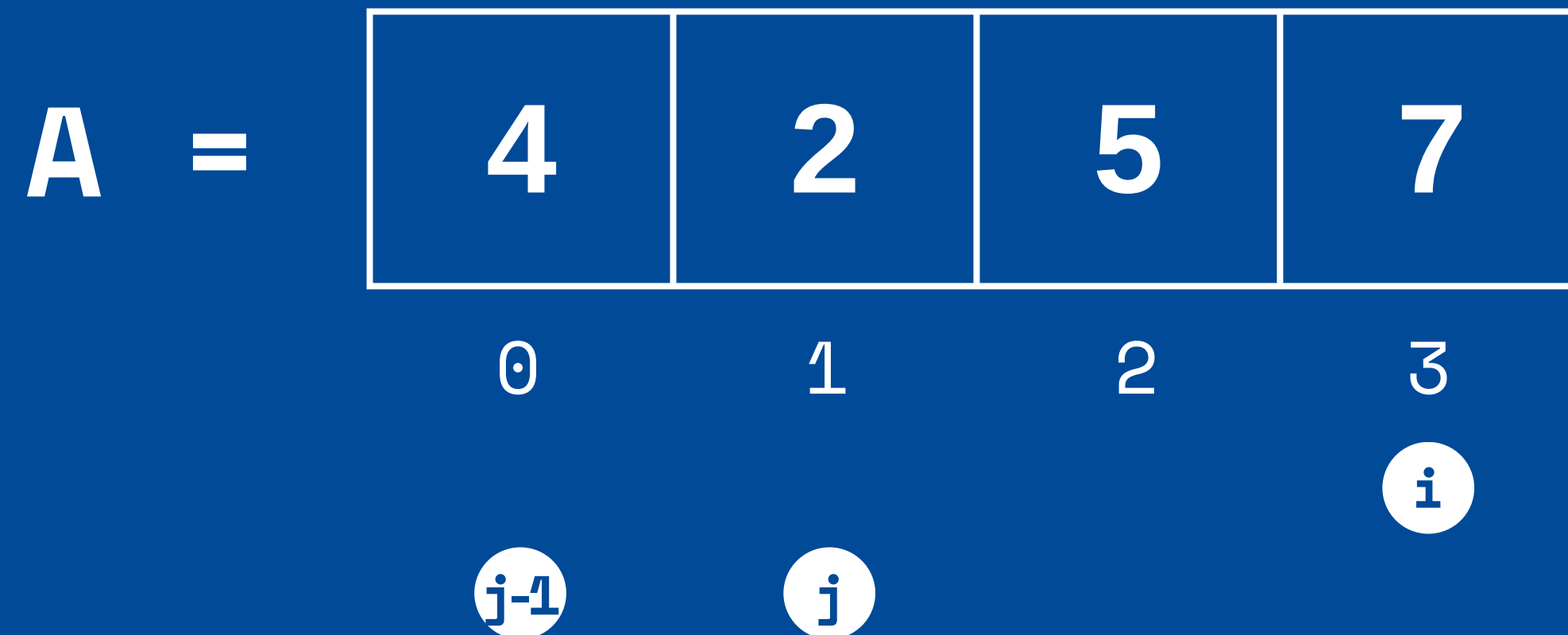


insertion sort



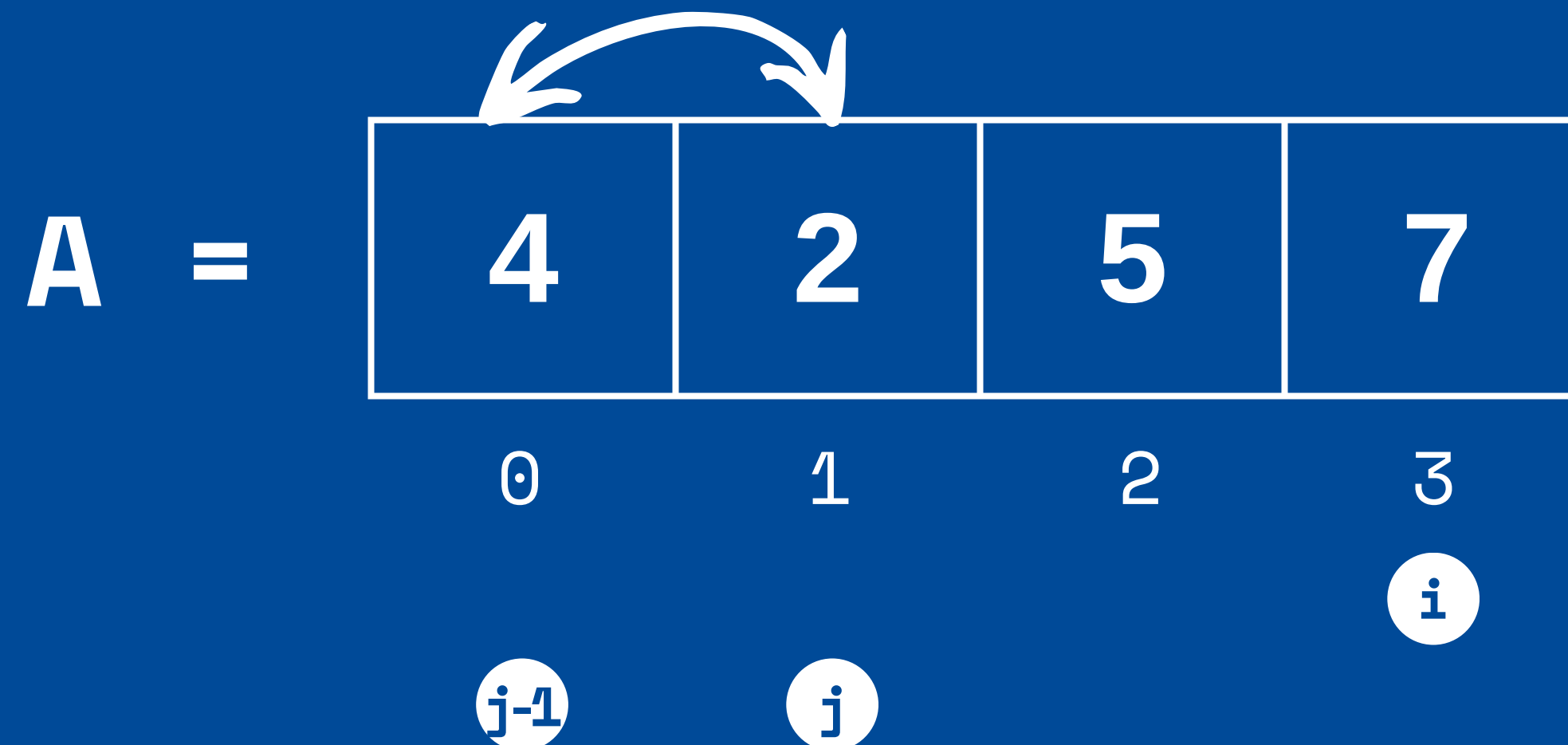
Se actualiza **j**: $j = j - 1 = 1$

insertion sort



Es el elemento $A[j] < A[j-1]$?

insertion sort



En este caso $2 < 4$, por lo que los intercambiamos

insertion sort

A =

2	4	5	7
---	---	---	---

0

1

2

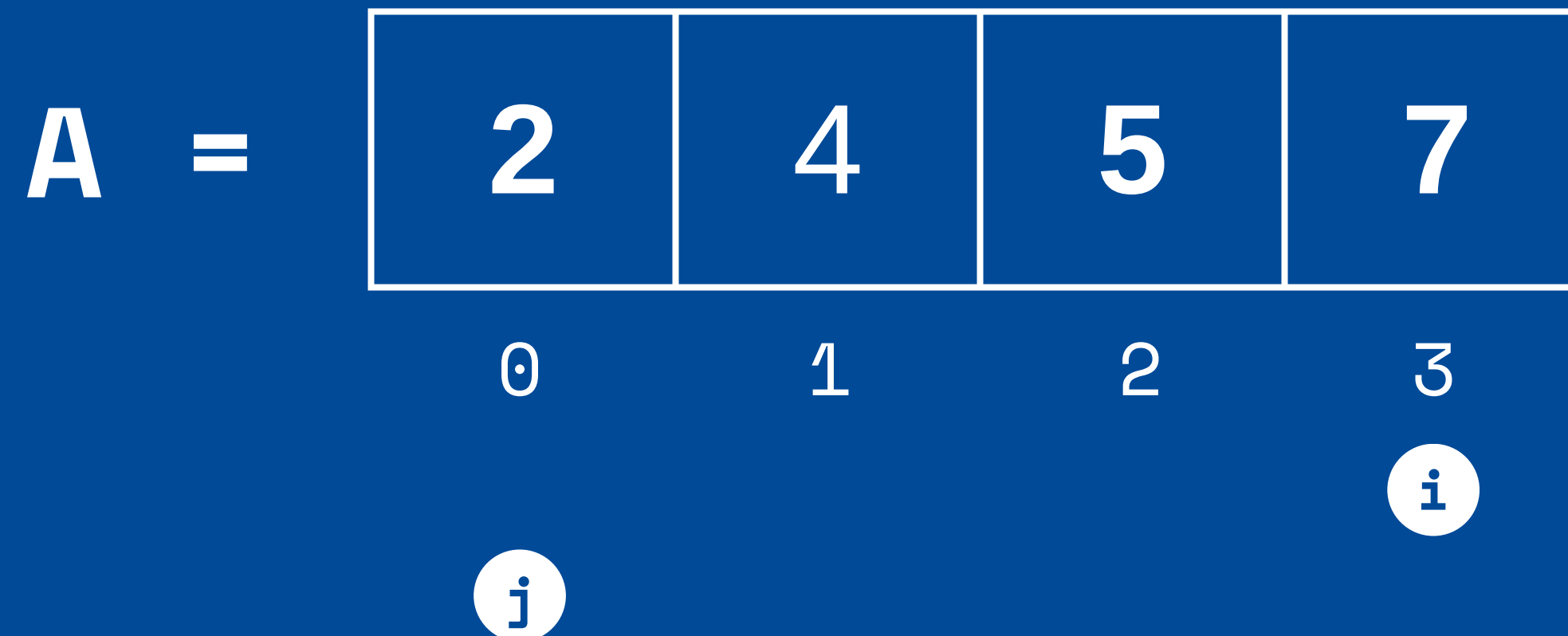
3

i

j-1

j

insertion sort



Se actualiza j : $j = j - 1 = 0$, lo que rompe el while

insertion sort

A =

2	4	5	7
0	1	2	3

Y se termina la ejecución del algoritmo,
quedando con el arreglo ordenado

Merge



Version not Inplace



Input: secuencias A
y B ordenadas

Output: secuencia C
ordenada

Memoria adicional: $O(n)$
Complejidad tiempo: $O(n)$

Merge(A,B):

1. Nueva secuencia vacia C
2. Sea a y b los primeros elementos de A y B respectivamente
3. Extraemos menor entre a y b de su secuencia
4. Si A y B no vacíos volvemos a 2
5. Concatenar a C la secuencia no vacía

return C



Merge



Version Inplace



Notamos que para la versión not in place utilizamos memoria adicional al usar una secuencia nueva para almacenar los elementos de A y B, por ende usamos $O(n)$ en memoria adicional, es decir, $|A| + |B| = n$. Por lo cual para la versión in place lo que se hace es usar el mismo espacio reservado a A y B, osea que la memoria adicional es $O(1)$, y luego se a de mover todos los datos mayores al insertado, lo cual claramente genera un costo en el tiempo al tener complejidad de $O(n^2)$.

Merge Sort



Version not Inplace



Input: Secuencia A

Output: Secuencia ordenada B

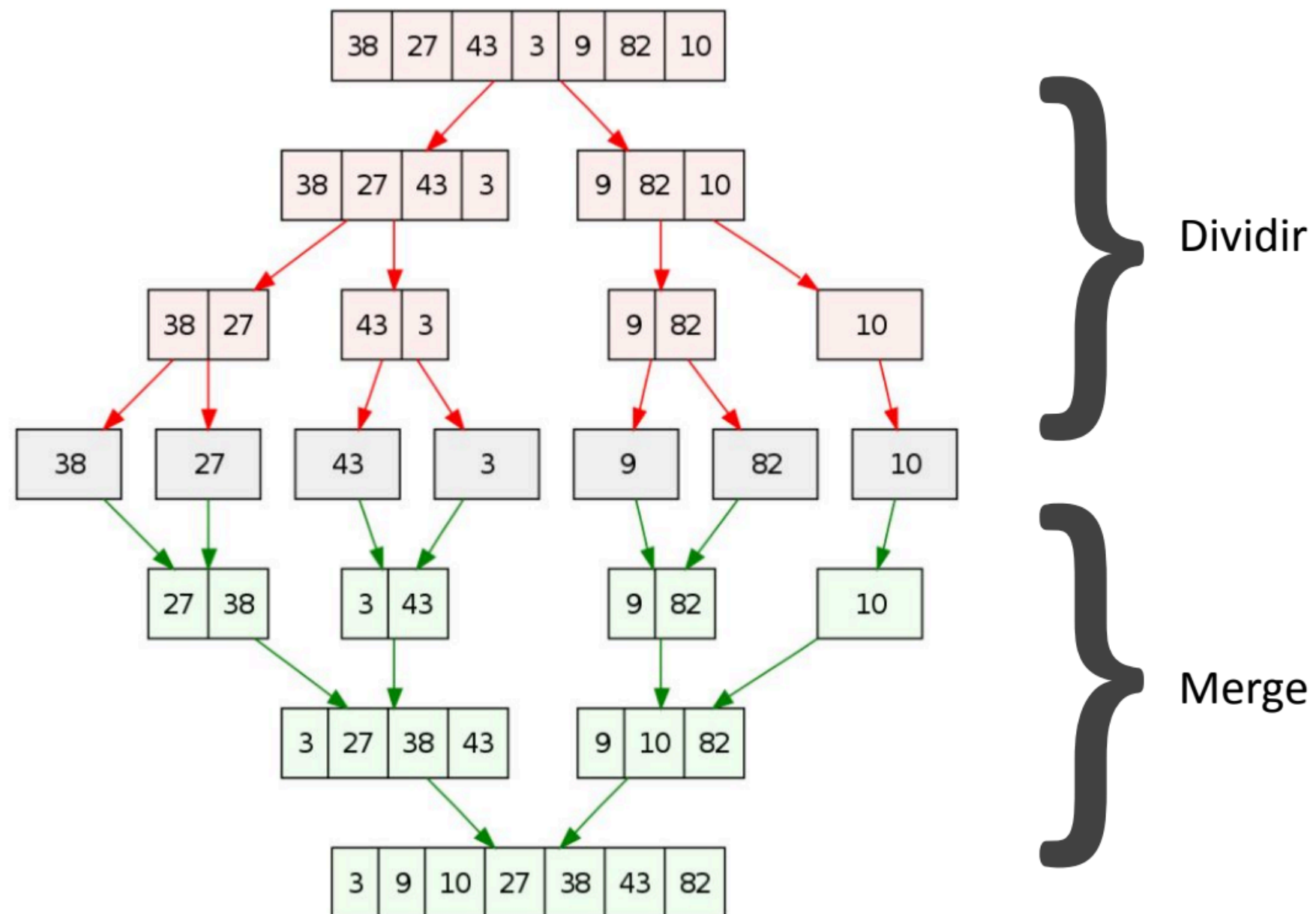
Qué estrategia algorítmica ocupa?

MergeSort (A):

```
1  if  $|A| = 1$  : return  $A$ 
2  Dividir  $A$  en  $A_1$  y  $A_2$ 
3   $B_1 \leftarrow \text{MergeSort}(A_1)$ 
4   $B_2 \leftarrow \text{MergeSort}(A_2)$ 
5   $B \leftarrow \text{Merge}(B_1, B_2)$ 
6  return  $B$ 
```

Merge Sort

Debido a la recursión el orden de los pasos no es el siguiente pero la ilustración no quita precisión sobre la idea principal y resultado del algoritmo



Complejidad

Mejor, promedio, peor caso

$O(n \log(n))$

Memoria adicional

$O(n)$

Ejercicio 1



La debilidad de los algoritmos de ordenación por comparación de elementos adyacentes, como `insertionSort`, en cuanto al número de de operaciones que ejecutan radica justamente en que sólo comparan e intercambian elementos adyacentes. Así, si tuviéramos un algoritmo que usara la misma estrategia de `insertionSort`, pero que comparara elementos que están a una distancia > 1 entre ellos, entonces podríamos esperar un mejor desempeño.

Adaptación Pregunta 3 - I1-2018-2





Dado el al algoritmo de ordenación por comparación `shellSort`

```
shellSort(A):  
gaps = {5,3,1}  
t = 0  
while t < 3:  
    gap = gaps[t]  
    i = gap  
    while i < A.length:  
        tmp = A[i]  
        j = i  
        while j >= gap and tmp < A[j-gap]:  
            A[j] = A[j-gap]  
            j = j - gap  
        A[j] = tmp  
        i = i + 1  
    t = t + 1
```

Adaptación Pregunta 3 - I1-2018-2





Calcule cuántas comparaciones entre elementos hace el algoritmo para ordenar el arreglo $A = [11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1]$. Luego analice cuantas comparaciones hace `insertionSort`. ¿Cuál es la relación entre estos dos algoritmos?

Adaptación Pregunta 3 - I1-2018-2



Ejercicio 1 - Solución



¿Dónde encontramos las comparaciones entre elementos del arreglo?

```
shellSort(A):  
    gaps = {5,3,1}  
    t = 0  
    while t < 3:  
        gap = gaps[t]  
        i = gap  
        while i < A.length:  
            tmp = A[i]  
            j = i  
            while j >= gap and tmp < A[j-gap]:  
                A[j] = A[j-gap]  
                j = j - gap  
            A[j] = tmp  
            i = i+1  
        t = t+1
```

Ejercicio 1 - Solución



Veamos para la primera iteración: $\text{gap} = 5$

$A = [11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1]$

```
i = gap
while i < A.length:
    tmp = A[i]
    j = i
    while j >= gap and tmp < A[j-gap]:
        A[j] = A[j-gap]
        j = j - gap
    A[j] = tmp
    i = i+1
t = t+1
```

$6 < 11 \rightarrow \text{true}$	$[6, 10, 9, 8, 7, \mathbf{11}, 5, 4, 3, 2, 1]$
$5 < 10 \rightarrow \text{true}$	$[6, \mathbf{5}, 9, 8, 7, 11, \mathbf{10}, 4, 3, 2, 1]$
$4 < 9 \rightarrow \text{true}$	$[6, 5, \mathbf{4}, 8, 7, 11, 10, \mathbf{9}, 3, 2, 1]$
$3 < 8 \rightarrow \text{true}$	$[6, 5, 4, \mathbf{3}, 7, 11, 10, 9, \mathbf{8}, 2, 1]$
$2 < 7 \rightarrow \text{true}$	$[6, 5, 4, 3, \mathbf{2}, 11, 10, 9, 8, \mathbf{7}, 1]$
$1 < 11 \rightarrow \text{true}$	$[6, 5, 4, 3, 2, \mathbf{1}, 10, 9, 8, 7, \mathbf{11}]$
$1 < 6 \rightarrow \text{true}$	$[5, 4, 3, 2, \mathbf{6}, 10, 9, 8, 7, 11]$



Ejercicio 1 - Solución

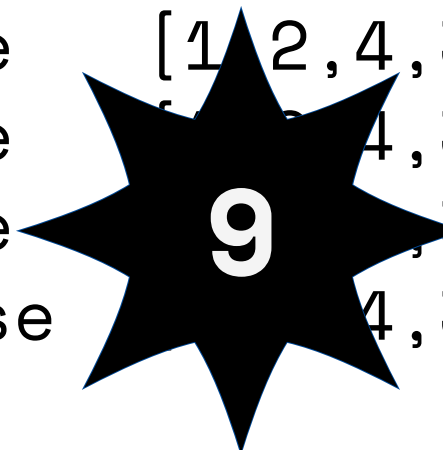


Segunda iteración: gap = 3

A = [1, 5, 4, 3, 2, 6, 10, 9, 8, 7, 11]

```
i = gap
while i < A.length:
    tmp = A[i]
    j = i
    while j >= gap and tmp < A[j-gap]:
        A[j] = A[j-gap]
        j = j - gap
    A[j] = tmp
    i = i+1
    t = t+1
```

3 < 1 → false	[1, 5, 4, 3, 2, 6, 10, 9, 8, 7, 11]
2 < 5 → true	[1, 2 , 4, 3, 5 , 6, 10, 9, 8, 7, 11]
6 < 4 → false	[1, 2, 4, 3, 5, 6, 10, 9, 8, 7, 11]
10 < 3 → false	[1, 2, 4, 3, 5, 6, 10, 9, 8, 7, 11]
9 < 5 → false	[1, 2, 4, 3, 5, 6, 10, 9, 8, 7, 11]
8 < 6 → false	[1, 2, 4, 3, 5, 6, 10, 9, 8, 7, 11]
7 < 10 → true	[1, 2, 4, 3, 5, 6, 7 , 9, 8, 10 , 11]
7 < 3 → false	[1, 2, 4, 3, 5, 6, 7, 9, 8, 10, 11]
11 < 9 → false	[1, 2, 4, 3, 5, 6, 7, 9, 8, 10, 11]



Ejercicio 1 - Solución



Tercera iteración: gap = 1

A = [1, 2, 4, 3, 5, 6, 7, 9, 8, 10, 11]

```
i = gap
while i < A.length:
    tmp = A[i]
    j = i
    while j >= gap and tmp < A[j-gap]:
        A[j] = A[j-gap]
        j = j - gap
    A[j] = tmp
    i = i+1
t = t+1
```

2 < 1 → false	[1, 2, 4, 3, 5, 6, 7, 9, 8, 10, 11]
4 < 2 → false	[1, 2, 4, 3, 5, 6, 7, 9, 8, 10, 11]
3 < 4 → true	[1, 2, 3 , 4 , 5, 6, 7, 9, 8, 10, 11]
3 < 2 → false	[1, 2, 3, 4, 5, 6, 7, 9, 8, 10, 11]
5 < 4 → false	[1, 2, 3, 4, 5, 6, 7, 9, 8, 10, 11]
6 < 5 → false	[1, 2, 3, 4, 5, 6, 7, 9, 8, 10, 11]
7 < 6 → false	[1, 2, 3, 4, 5, 6, 7, 9, 8, 10, 11]
9 < 7 → false	[1, 2, 3, 4, 5, 6, 7, 9, 8, 10, 11]
8 < 9 → true	[1, 2, 3, 4, 5, 6, 7, 8 , 9 , 10, 11]
8 < 7 → false	[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]
10 < 9 → false	[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]
11 < 10 → false	[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]

12

Ejercicio 1 - Solución



Total comparaciones shellSort

$$7 + 9 + 12 = 28$$

Total comparaciones insertionSort

$$55$$

Ejercicio 2



Escriba una versión de MergeSort que no respete el orden original de los datos e indique que parte cambiar para que si lo respete

Adaptación Pregunta 1b - I1-2017-1



Ejercicio 2 - Solucion



Cómo vemos esto en MergeSort?

input : Secuencia A

output: Secuencia ordenada B

MergeSort (A):

```
1  if  $|A| = 1$  : return  $A$ 
2  Dividir  $A$  en mitades  $A_1$  y  $A_2$ 
3   $B_1 \leftarrow \text{MergeSort}(A_1)$ 
4   $B_2 \leftarrow \text{MergeSort}(A_2)$ 
5   $B \leftarrow \text{Merge}(B_1, B_2)$ 
6  return  $B$ 
```

input : Secuencias ordenadas A y B

output: Nueva secuencia ordenada C

Merge(A, B):

```
1  Iniciamos  $C$  vacía
2  Sean  $a$  y  $b$  los primeros elementos de  $A$  y  $B$ 
3  Extraer de su secuencia respectiva el menor entre  $a$  y  $b$ 
4  Insertar el elemento extraído al final de  $C$ 
5  Si quedan elementos en  $A$  y  $B$ , volver a línea 2
6  Concatenar  $C$  con la secuencia que aún tenga elementos
7  return  $C$ 
```

Ejercicio 2 - Solucion



Cómo vemos esto en MergeSort?

Todo va a depender de como se decida cuál elemento extraer en caso de empate.

Al ponerlo en código, si la condición fuese

`if a < b: extraer a`

podemos obtener la versión de MergeSort que nos piden

input : Secuencias ordenadas A y B
output: Nueva secuencia ordenada C

Merge(A, B):

- 1 Iniciamos C vacía
- 2 Sean a y b los primeros elementos de A y B
- 3 **Extraer de su secuencia respectiva el menor entre a y b**
- 4 Insertar el elemento extraído al final de C
- 5 Si quedan elementos en A y B , volver a línea 2
- 6 Concatenar C con la secuencia que aún tenga elementos
- 7 **return** C

Ejercicio 2 - Solucion



Cómo vemos esto en MergeSort?

Para transformarla debemos asegurarnos de priorizar el orden original del input en caso de empate.

En Merge, siempre los elementos de a iban antes en el orden original, por lo que la condición debiese ser

`if a <= b: extraer a`

input : Secuencias ordenadas A y B
output: Nueva secuencia ordenada C

Merge(A, B):

- 1 Iniciamos C vacía
- 2 Sean a y b los primeros elementos de A y B
- 3 Extraer de su secuencia respectiva el menor entre a y b
- 4 Insertar el elemento extraído al final de C
- 5 Si quedan elementos en A y B , volver a línea 2
- 6 Concatenar C con la secuencia que aún tenga elementos
- 7 **return** C

Ejercicio 2 - Solucion



Por qué nos importaría?

Tomando el ejemplo de la ayudantía pasada, supongamos que queremos ordenar las plantas siempre tal que queden los girasoles primero, luego los lanzaguisantes y al final las nueces.



0



1



2

Podemos asignarles un número de manera que al ordenarlos con un algoritmo coincida con el orden arbitrario que nos piden.

Ejercicio 2 - Solucion



Pero qué pasa si además cada planta tiene potencia de ataque diferente?

Si tenemos las plantas ordenadas por potencia:



1

9



2

9



0

7



1

5



2

2



2

1

Ejercicio 2 - Solucion



Si reordenamos la lista podemos obtener este orden, que sigue la regla del tipo de planta, pero hay algunas con mayor potencia que quedan al final



0

7



1

5



1

9



2

9



2

1



2

2

Si nos preocupamos de respetar el orden original del arreglo
obtendremos un orden que respete el criterio de tipo de planta y
deje primero las con mayor potencia



1

9



2

9



0

7



1

5



2

2



2

1



0

7



1

9



1

5



2

9



2

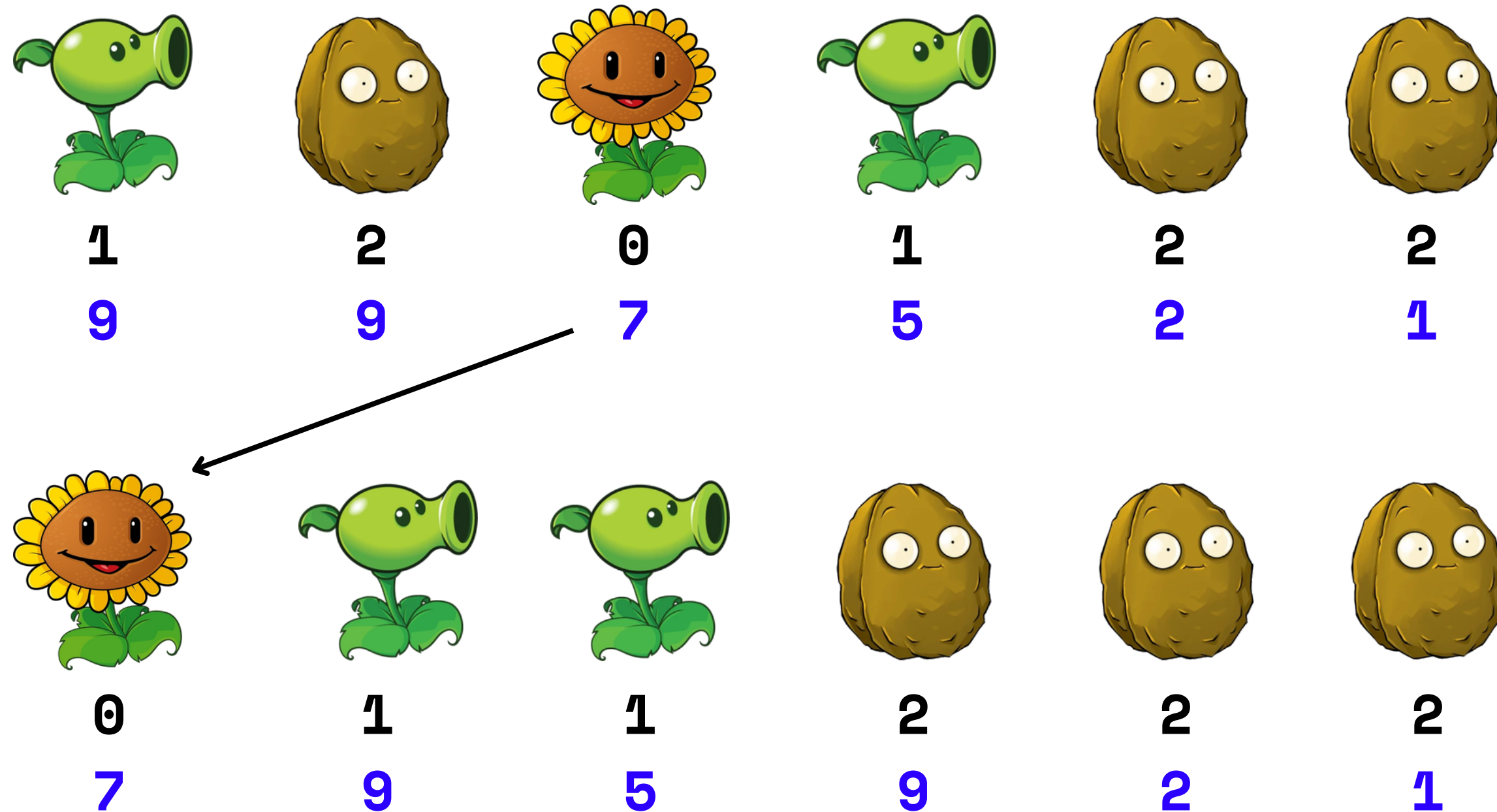
2



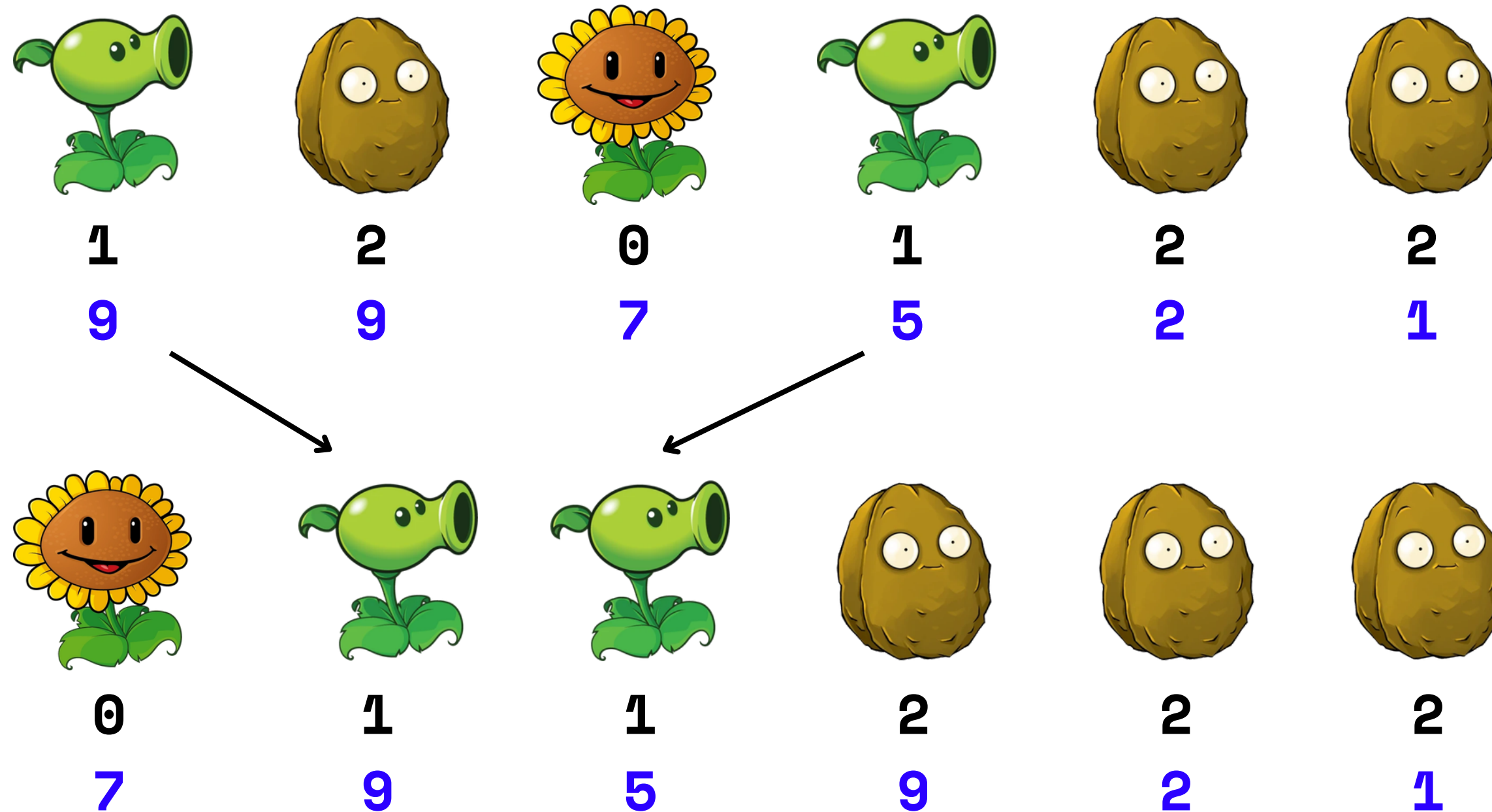
2

1

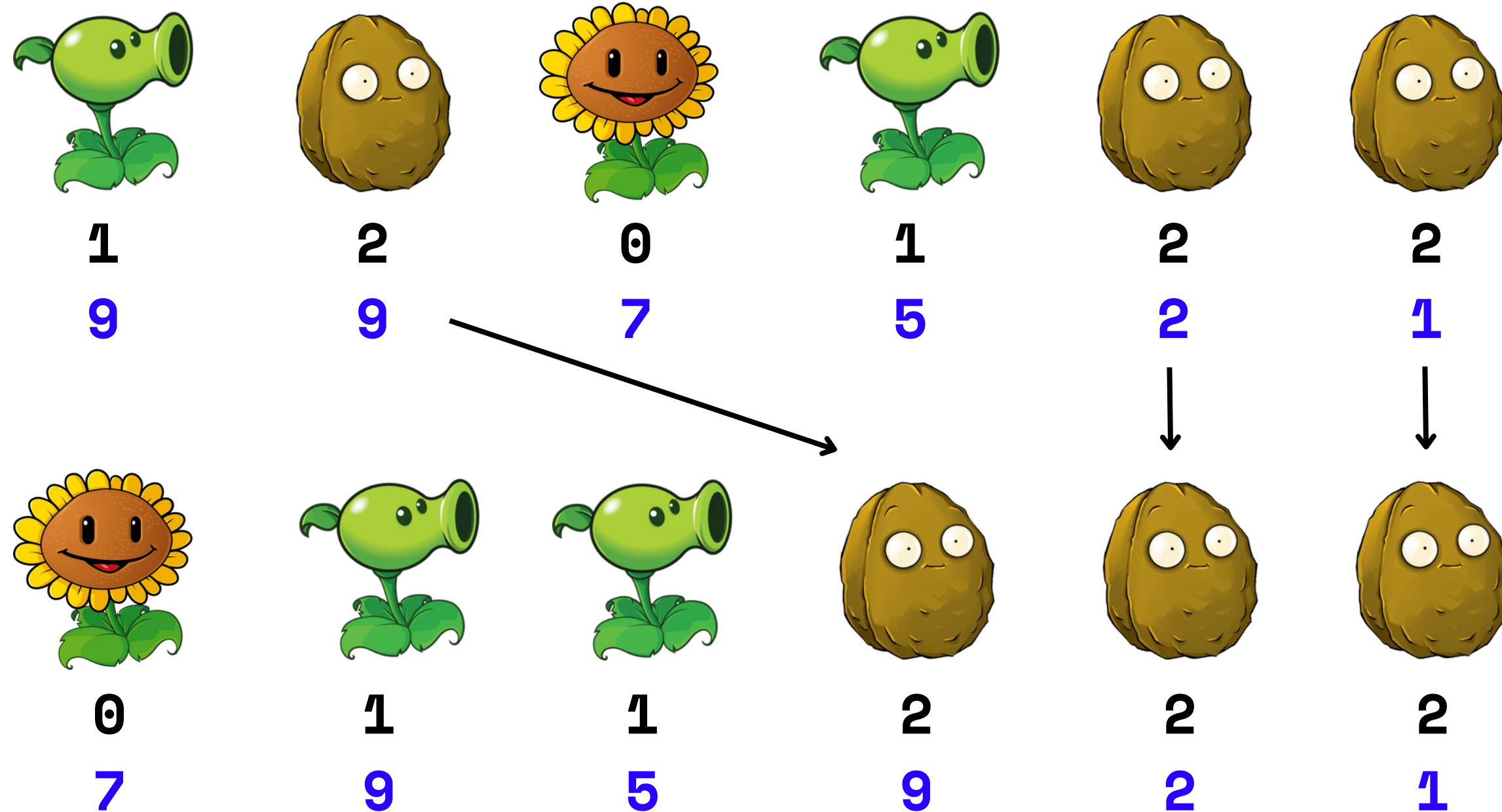
Si nos preocupamos de respetar el orden original del arreglo obtendremos un orden que respete el criterio de tipo de planta y deje primero las con mayor potencia



Si nos preocupamos de respetar el orden original del arreglo obtendremos un orden que respete el criterio de tipo de planta y deje primero las con mayor potencia



Si nos preocupamos de respetar el orden original del arreglo obtendremos un orden que respete el criterio de tipo de planta y deje primero las con mayor potencia



Qué es un algoritmo estable?

Un algoritmo estable es aquel que mantiene el orden del input original cuando el algoritmo de comparaciones no es capaz de distinguir dos o más elementos. Va a garantizar que cuando dos o más objetos son iguales, los ordenará de la misma forma en que estaban originalmente.

Feedback

